PowerUp Session 02 Homework — Four New Endpoints

Goal

In class you read every route in `server/index.js`. Now you **write** routes yourself. You will add four new `GET` endpoints to the B4F Hub server — the same four the main Bootcamp class added at this point in the course. Next Saturday starts by solving this homework together and connecting two of these endpoints to the React app, so finish it before then.

Expect about **60–120 minutes**.

Rules

- Work on **your own branch** (`student-<your-github-username>`), never on `main`.
- Add everything to **`server/index.js`** — the same single-file structure we read in class. Do **not** create new folders or split the server into other files.
- Use only what you saw in class: `app.get`, `req.params`, `Number(...)`, `.find(...)` / `.filter(...)`, `res.json(...)`, `res.status(...).json(...)`, an early `return`.
- Do **not** use: `req.query`, any new npm package, a database, new middleware.
- Do **not** change how any existing endpoint behaves, and do not change `client/`, `practice-client/` or `docs/`.
- After every change, stop the server (Ctrl+C) and run `npm start` again — it does not reload by itself.

Test with the browser (every endpoint here is a `GET`, so you can open the URL directly), with curl, or with Postman / Thunder Client.

CORE — get one record by id

These two endpoints have the same shape as the existing `PATCH` routes: read an id from the URL, look it up, answer **404** if nothing matches.

C1. `GET /api/opportunities/:id`

Method	`GET`
Full path	`/api/opportunities/:id` — for example `/api/opportunities/101`
Input	the id in the URL → `req.params.id` (a string — convert it to a number)
Request body	none
Success	**200** — the one opportunity object with that id
Error	**404** — `{ "error": "No opportunity found with id 999." }` (use the real id in the message)

Put it next to the other opportunity routes.

Test:

BASH

```
curl http://localhost:3001/api/opportunities/101
curl http://localhost:3001/api/opportunities/999
```

Expected: the first returns the opportunity "Junior Frontend Developer" (200); the second returns the error (404).

C2. `GET /api/posts/:id`

Method	`GET`
Full path	`/api/posts/:id` — for example `/api/posts/1`
Input	`req.params.id` (convert it to a number)
Request body	none
Success	**200** — the one post object with that id
Error	**404** — `{ "error": "No post found with id 999." }`

Put it next to the other post routes.

Test:

BASH

```
curl http://localhost:3001/api/posts/1
curl http://localhost:3001/api/posts/999
```

Expected: post 1 by "Mona Al-Sayed" (200); then the error (404).

STRETCH — a health check

S1. `GET /api/health`

A "health" endpoint lets anyone check quickly that the server is running and see how much data it holds.

Method	`GET`
Full path	`/api/health`
Input	none — no route parameter, no body
Success	**200** — `{ "status": "ok", "opportunities": 20, "posts": 18 }`
Error	none — this endpoint always succeeds

The two numbers must be the **current** lengths of the server's `opportunities` and `posts` arrays — not numbers you type by hand.

Test:

BASH

```
curl http://localhost:3001/api/health
curl -X POST http://localhost:3001/api/posts -H "Content-Type: application/json" -d
'{"content": "Testing health", "category": "event"}'
curl http://localhost:3001/api/health
```

Expected: `"posts": 18`, then a new post is created (201), then `"posts": 19`. Restart the server and call health again: back to 18. Why?

CHALLENGE — filter by type, with a 400

X1. `GET /api/opportunities/type/:type`

Method	`GET`
Full path	`/api/opportunities/type/:type` — for example `/api/opportunities/type/internship`
Input	`req.params.type` — one of `job`, `internship`, `scholarship`, `volunteer`
Request body	none
Success	**200** — an array with every opportunity of that type
Error	**400** — when `:type` is not one of the four: `{ "error": "type must be one of: job, internship, scholarship, volunteer." }`

Keep the four allowed types in a constant, the same way the server already keeps `ALLOWED_CATEGORIES` for posts.

An **empty array is a correct 200 answer**, not an error: "there are no opportunities of this type right now" and "this is not a real type" are different problems, so they get different status codes.

Test:

BASH

```
curl http://localhost:3001/api/opportunities/type/internship
curl http://localhost:3001/api/opportunities/type/volunteer
curl http://localhost:3001/api/opportunities/type/remote
```

Expected: an array of internships (200); an array of volunteer roles (200); the error (400) — `remote` is a work mode, not a type.

Think about it: your server now has both `/api/opportunities/:id` and `/api/opportunities/type/:type`. Does `/api/opportunities/type/internship` ever reach the `:id` route? Test it, and explain why in one or two sentences. (Hint: count the parts of each path between the `/` characters.)

Check everything still works

1. `node --check index.js` (inside `server/`) reports no errors.
2. All the tests above give the expected status codes.
3. The old endpoints still work: `GET /api/opportunities`, Apply (`PATCH /api/opportunities/:id`), `GET /api/posts`, creating and liking a post.
4. With the server running, start `client/` (`npm run dev`) — the app works exactly as before.

Submitting

Commit and push to your own `student-<your-github-username>` branch. Tell your instructor which levels you finished (CORE, STRETCH, CHALLENGE), and bring your questions to next Saturday.

Done when

- CORE: both "get by id" endpoints answer 200 / 404 as described.
- STRETCH: `/api/health` answers 200 with the real, current counts.
- CHALLENGE: the type filter answers 200 (including an empty array) / 400 as described.
- Everything is inside `server/index.js`, and every existing endpoint still works.